



ML-Based XSS Honeypot

A Hybrid Machine Learning & Rule-Based Honeypot for Detecting, Monitoring
and Analyzing Reflected & Stored Cross-Site Scripting Attacks

Python 3.x

Flask 3.0

scikit-learn

SQLite

Educational Use

98.82%

ACCURACY

99%

PRECISION

99%

RECALL

2,544

PAYLOADS

Table of Contents

1. Project Overview	3
2. Objectives & Problem Statement	3
3. Hybrid XSS Detection Engine	4
4. Machine Learning Model	4
5. System Architecture	5
6. Main Features	5
7. Technology Stack	6
8. Installation Guide	6
9. Running the Application	7
10. Training the Machine Learning Model	7
11. Testing Strategy	8
12. Results	8
13. Screenshots	9
14. Security & Ethical Use	11
15. Limitations & Future Improvements	11
16. Research Context & Author	12
17. License & Disclaimer	12

1. Project Overview

The **ML-Based XSS Honeypot** is a web-based cybersecurity research system designed to detect, capture, monitor, and analyze **Reflected** and **Stored Cross-Site Scripting (XSS)** attacks within a controlled, isolated environment.

It combines **Machine Learning classification** with traditional **Keyword/Regex detection** to form a hybrid threat detection engine — rather than relying on a single method, submitted payloads are analyzed through multiple detection layers and logged for further security research.

This project was developed as a **Computer Researching Project** for the **Pearson BTEC HND in Computing** programme at **CINEC Campus**.

2. Objectives & Problem Statement

Objectives

- Design a controlled honeypot environment for Reflected & Stored XSS detection
- Implement a hybrid detection engine combining ML + Keyword/Regex
- Capture, monitor and analyze malicious payloads and attacker behavior
- Provide an administrator dashboard for live threat monitoring
- Enrich attack data with threat intelligence (GeoIP, reverse DNS, bot detection)
- Deliver real-time alerts via Telegram / Discord / Slack
- Support cybersecurity education, research, and secure development practices

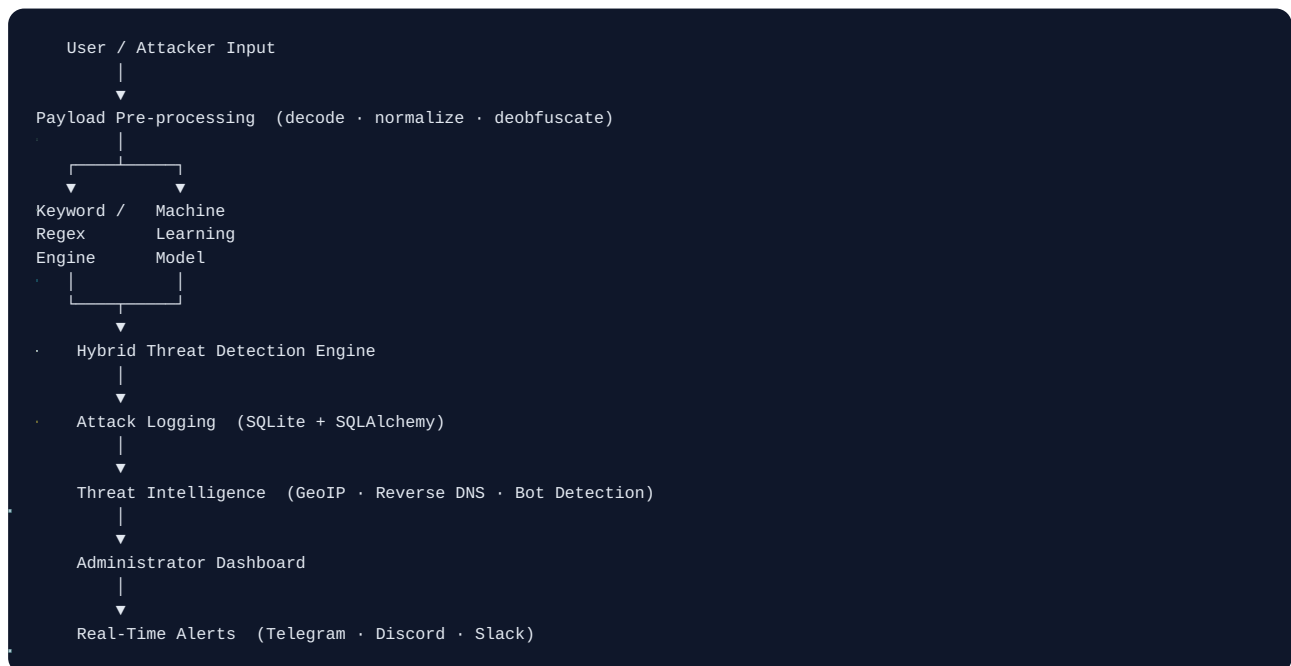
Problem Statement

Traditional web security controls often rely purely on predefined signatures, which struggle against:

- Modified or obfuscated XSS payloads
- Previously unseen attack patterns
- Suspicious inputs that don't exactly match known signatures
- Limited visibility into attacker behavior over time

This project addresses that gap by combining **honeypot technology + rule-based detection + Machine Learning + attack logging + threat intelligence** inside a controlled research environment.

3. Hybrid XSS Detection Engine



Keyword / Regex Detection

Fast rule-based matching against known XSS keywords, script tags, event-handler attributes, and regex signatures for recognizable payload structures.

Machine Learning Detection

Text-based classification using:

- **TF-IDF** feature extraction
- **Logistic Regression** classifier

- Trained on a labeled dataset of malicious/benign payloads

Hybrid Decision

Results from both engines are merged in the **Hybrid Threat Detection Engine**, giving broader coverage than either approach alone — including a confidence score per verdict.

4. Machine Learning Model

Dataset Composition

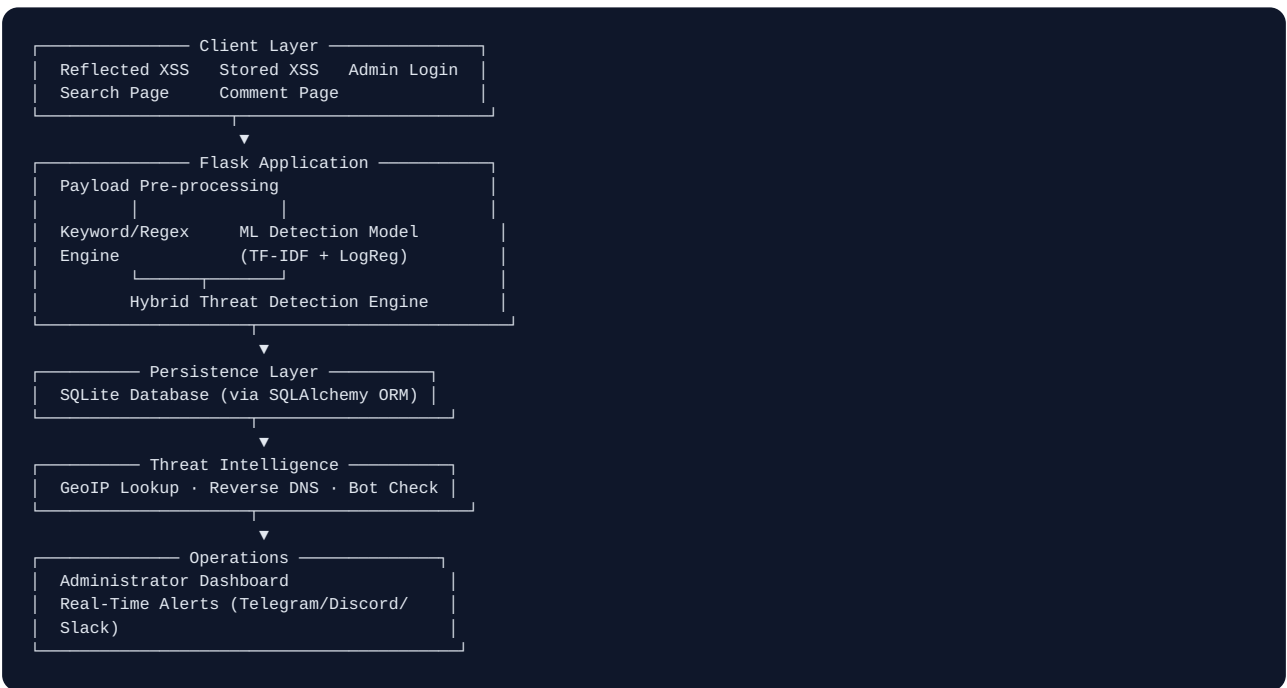
Dataset Category	Samples
Malicious XSS Payloads	1,272
Benign Inputs	1,272
Total	2,544

Model Performance

98.82% ACCURACY	99% PRECISION	99% RECALL	99% F1-SCORE	1.00 ROC-AUC
--------------------	------------------	---------------	-----------------	-----------------

These figures reflect the project's documented evaluation results and should not be interpreted as guaranteed performance against all real-world XSS attacks — highly obfuscated or novel payloads may still evade detection.

5. System Architecture



The implementation uses a **Flask** application, **scikit-learn** for ML classification, **SQLite + SQLAlchemy** for persistence, external **threat intelligence** services, and **webhook-based** notification channels.

6. Main Features

XSS Honeypot Interfaces

- Reflected XSS testing interface
- Stored XSS testing interface
- Controlled research environment
- Payload capture & analysis

Attack Logging

- Payload, method, timestamp
- Detection method & result
- Confidence score
- Source IP, ISP, geolocation

Machine Learning Detection

- TF-IDF feature extraction
- Logistic Regression classification
- Malicious / benign classification
- Confidence-scored predictions

Threat Intelligence

- GeoIP lookup
- Reverse DNS lookup
- Bot / automation detection

Rule-Based Detection

- Keyword detection
- Regex pattern matching
- Suspicious pattern identification

Admin Dashboard & Alerts

- Real-time monitoring & analytics
- Attack trends & origin distribution
- Telegram / Discord / Slack alerts

7. Technology Stack

Python

Flask

HTML5

CSS3

JavaScript

SQLite

SQLAlchemy

scikit-learn

Pandas

NumPy

Git

VS Code

Technology	Purpose
Python	Core application development
Flask	Web application framework
HTML / CSS / JS	Frontend structure, styling & interaction
SQLite + SQLAlchemy	Attack log storage & ORM
Scikit-learn	ML model training & inference
Pandas / NumPy	Dataset preprocessing
TF-IDF + Logistic Regression	Text feature extraction & classification
Regex	XSS pattern detection
GeoIP / Reverse DNS	Threat intelligence enrichment
Telegram / Discord / Slack	Real-time alert delivery

Requirements

- Windows 10/11, macOS, or Linux
- Python 3.x
- pip
- A modern browser (Chrome / Edge)
- VS Code (*optional but recommended*)

8. Installation Guide

1. Clone the repository

```
git clone https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git
cd YOUR-REPOSITORY
```

2. Create a virtual environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

macOS / Linux:

```
source venv/bin/activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Configure environment variables

Create a `.env` file in the project root (never commit real secrets):

```
SECRET_KEY=change_this_to_a_random_secret
DATABASE_URL=sqlite:///honeypot.db
ADMIN_PASSWORD=change_this_password
DISCORD_WEBHOOK_URL=
SLACK_WEBHOOK_URL=
```

Add `.env` and `*.db` to your `.gitignore` before pushing — don't publish real credentials or captured attack data.

9. Running the Application

Start the Flask server:

```
python app.py
```

The app runs on:

```
http://127.0.0.1:5000
```

Testing the Honeypot

Step	Action
1	Open the home page at <code>http://127.0.0.1:5000</code>
2	Submit test payloads on the Reflected XSS Search Page

3	Submit test payloads on the Stored XSS Comment Page
4	Payloads are processed by the hybrid detection engine automatically
5	Log in at /admin/login (or /login) and open /dashboard to review results, detection method, confidence score, source info and threat intelligence

10. Training the Machine Learning Model

The classifier (TF-IDF + Logistic Regression) is trained by `train_model.py`, producing the artifacts that `app.py` loads at runtime.

1. Prepare the payload dataset

Place labeled `.txt` payload files inside a `payloads/` directory in the project root (one payload per line):

```
payloads/
├─ attribute_based_xss_payloads.txt
├─ blind_xss_payloads.txt
├─ angularjs.txt
└─ ...
```

2. Run the trainer

```
python train_model.py
```

The script will:

1. Load and deduplicate malicious payloads from `payloads/*.txt`
2. Generate / augment a benign dataset
3. Split the data 80% train / 20% test (stratified)
4. Fit a **TF-IDF vectorizer**
5. Train a **Logistic Regression** classifier
6. Evaluate on the held-out test set + run 5-fold cross-validation
7. Run a quick inference smoke test
8. Save trained artifacts

3. Output artifacts

```
ml_model/
├─ xss_model.pkl           # Trained Logistic Regression classifier
├─ tfidf_vectorizer.pkl    # Fitted TF-IDF vectorizer
└─ training_report.json    # Accuracy, precision, recall, F1, ROC-AUC, confusion matrix
```

Once training completes, simply (re)start the Flask app — `app.py` automatically loads the model and vectorizer from `ml_model/` and uses them alongside the regex-based detector:

```
python app.py
```

11. Testing Strategy

Level	Coverage
Unit Testing	Payload pre-processing, Keyword/Regex detection, ML detection, Hybrid detection, threat intelligence, alerts, database logging
Integration Testing	Full pipeline: submission → pre-processing → detection → hybrid decision → logging → threat intel → dashboard → alert
System Testing	Functional & non-functional requirements: detection accuracy, logging, admin auth, dashboard monitoring, threat intelligence, real-time alerts, performance, reliability, usability, maintainability, security

All documented system testing results marked the defined functional and non-functional requirements as **passed**.

12. Results

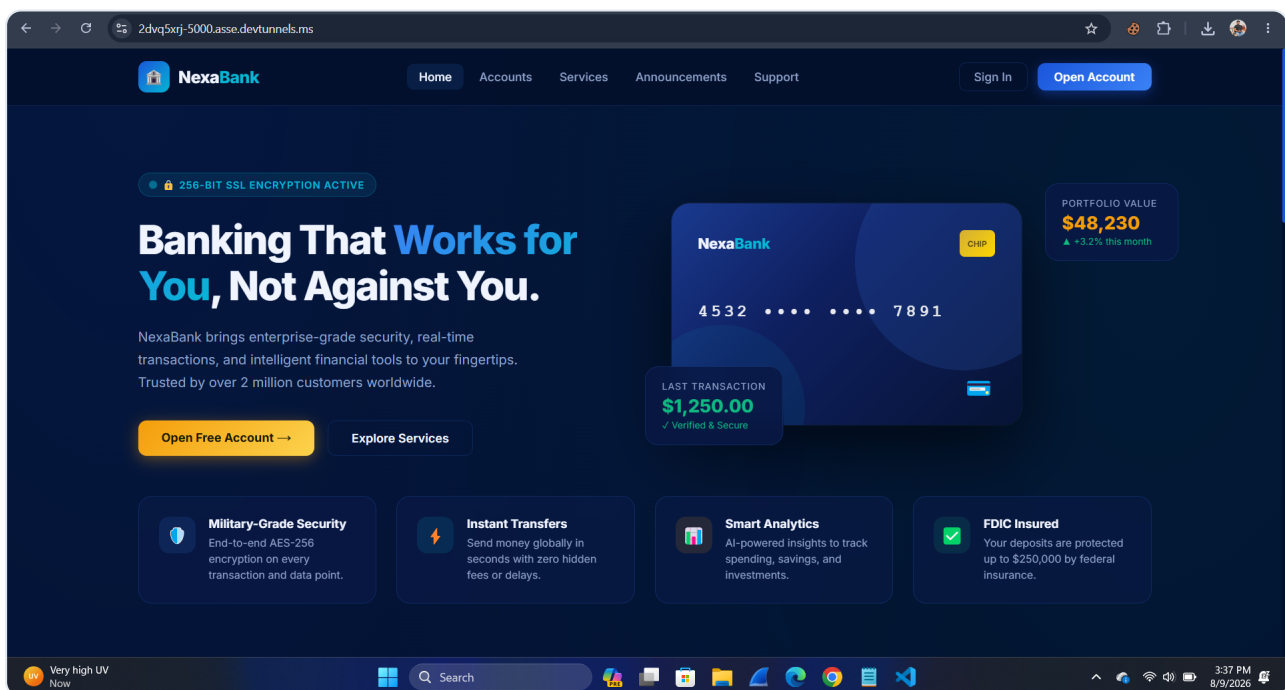
```
Dataset Size      : 2,544 payloads
Malicious Samples : 1,272
Benign Samples    : 1,272
```

```
Accuracy      : 98.82%
Precision     : 99%
Recall        : 99%
F1-Score      : 99%
ROC-AUC       : 1.00
```

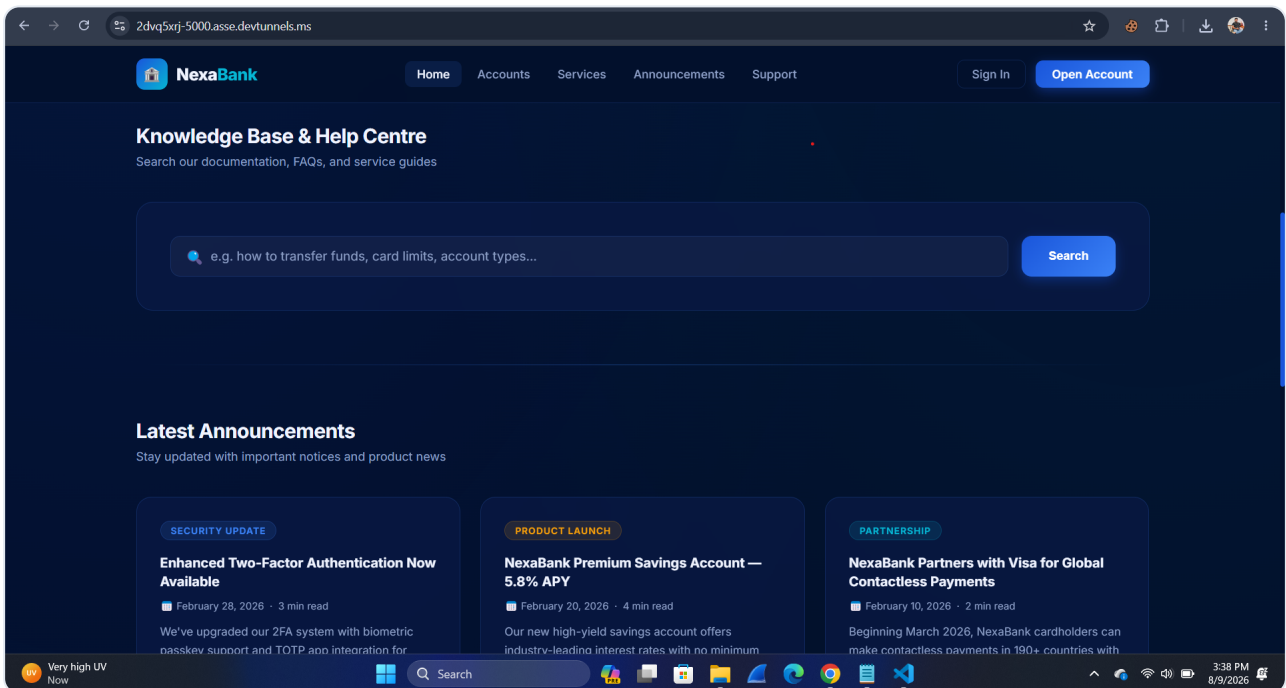
The system successfully:

- Detects Reflected and Stored XSS attacks
- Combines ML with Keyword/Regex detection in a hybrid engine
- Captures, logs and enriches malicious payload data
- Monitors attacks through a live administrator dashboard
- Delivers real-time security alerts

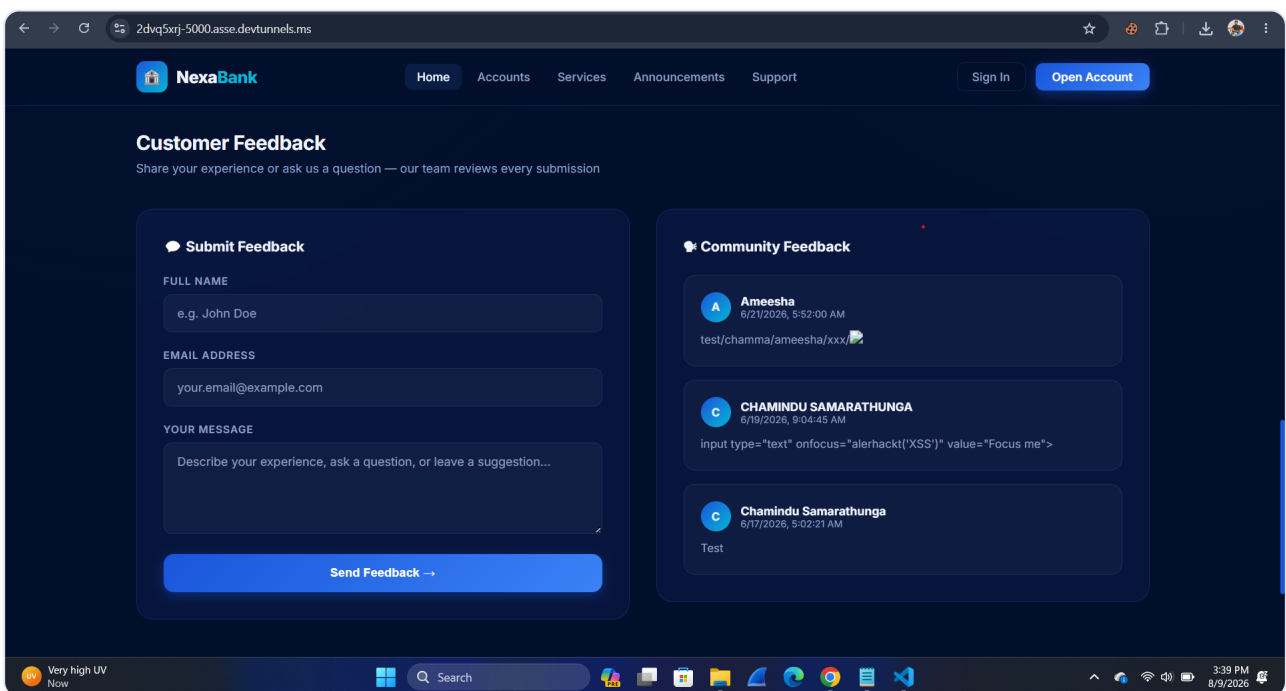
13. Screenshots



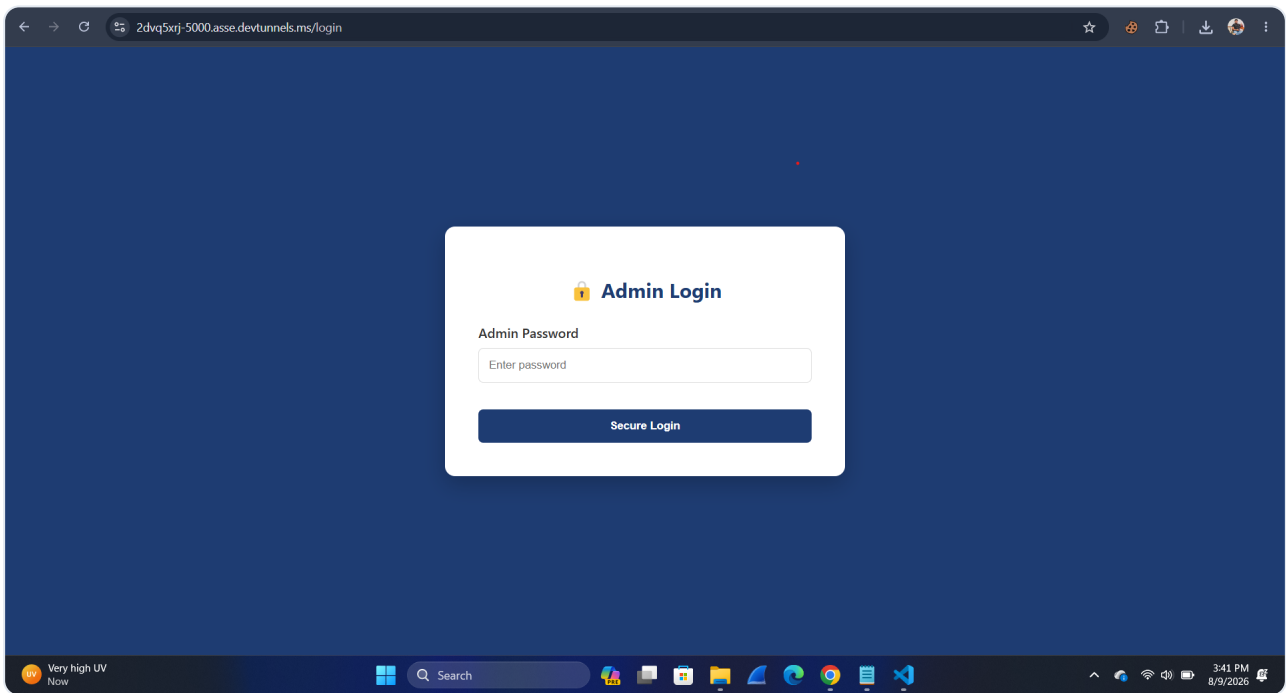
Home Page



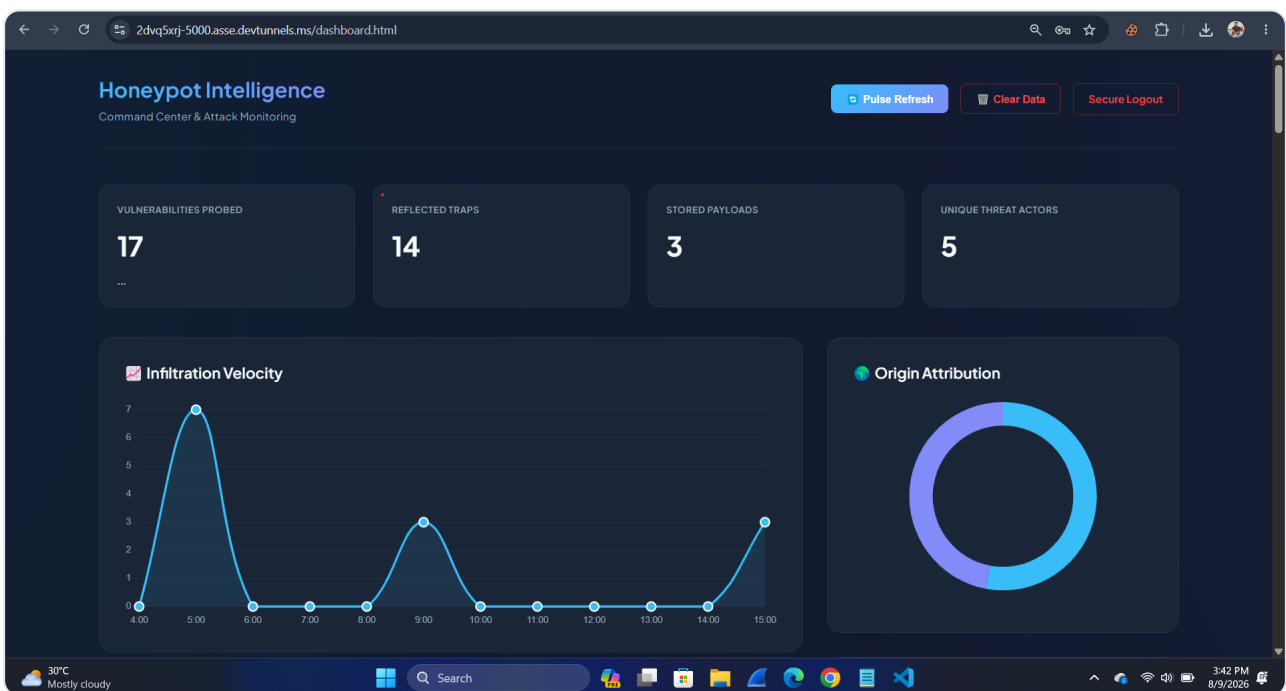
Reflected XSS Interface



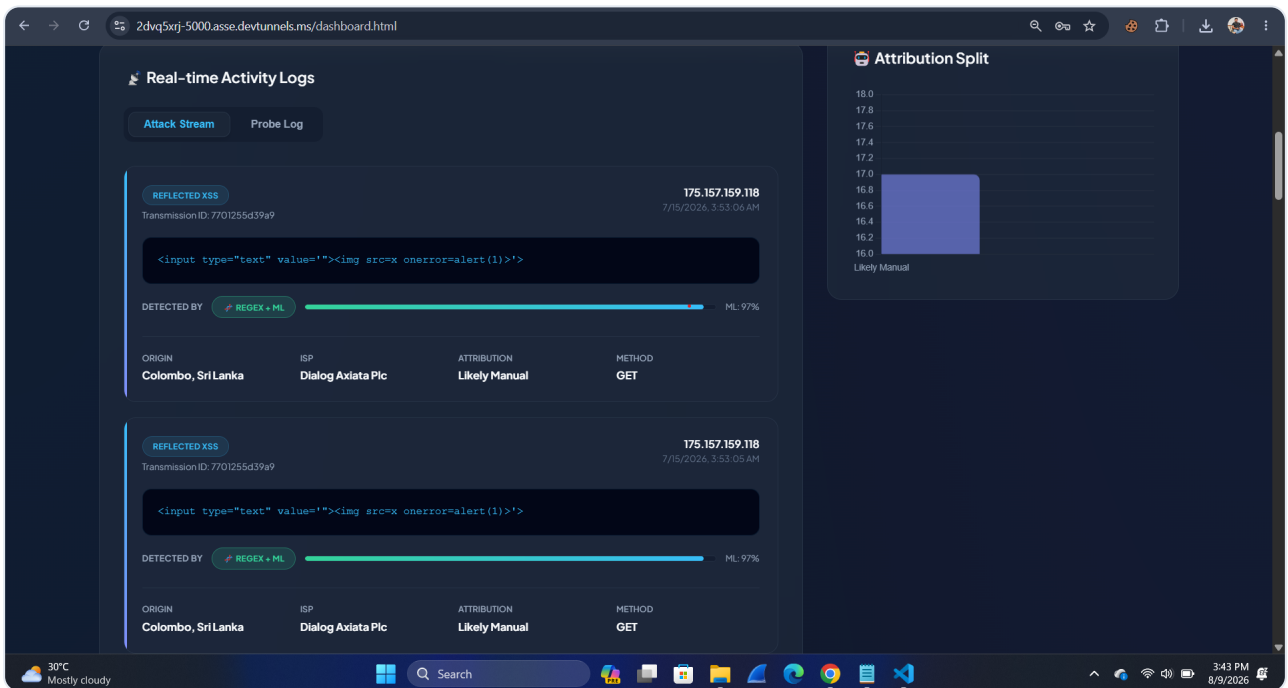
Stored XSS Interface



Administrator Login



Administrator Dashboard



Attack Details

14. Security & Ethical Use

This project is intended for **cybersecurity education, academic research, controlled security testing, honeypot research, and ML research**.

The honeypot is designed to run in a **controlled, isolated environment** and is **not** intended to interact with real production systems, real users, or sensitive organizational data.

Only use this project against systems you own or have explicit authorization to test. Do not deploy the intentionally vulnerable components to a public production environment without appropriate isolation and security controls.

15. Limitations & Future Improvements

Limitations

- Focuses primarily on Reflected & Stored XSS (SQLi, CSRF, RCE are out of scope)
- ML model trained on a limited dataset
- Highly obfuscated or previously unseen payloads may evade detection
- Evaluated in a controlled, not production, environment
- Threat intelligence depends on external service availability



Future Improvements

Area	Plan
ML	Larger/more diverse datasets, better obfuscation handling, fewer false positives, additional algorithms
Detection	Extend to SQLi, CSRF, RCE, LFI
Threat Intel	More sources, deeper attacker profiling
SIEM	Integrate with SIEM platforms for centralized monitoring
Automated Response	Incident-handling automation
Deployment	Controlled real-world deployment for richer datasets

16. Research Context & Author

```
Cybersecurity
├── Web Application Security
├── Cross-Site Scripting
├── Honeypot Technology
├── Machine Learning
├── Threat Detection
└── Threat Intelligence
```

Programme	Pearson BTEC HND in Computing
Institution	CINEC Campus
Supervisor	Mrs. Eshandhi Aththanayaka
Researcher	K. Chamindu Kawshik Vichakshana Samarathunga

Author

K. Chamindu Kawshik Vichakshana Samarathunga

Computer Researching Project · Pearson BTEC HND in Computing · CINEC Campus

Cybersecurity

Web Application Security

Security Operations

Threat Detection

Machine Learning

Ethical Hacking

Threat Intelligence

Secure Software Development

17. License & Disclaimer

License

This project is intended primarily for **academic, educational, and cybersecurity research purposes**. Add a `LICENSE` file that reflects how you want others to use, modify, and distribute the code before making the repository public.

Disclaimer

Developed for **authorized cybersecurity research, education, and controlled testing** only. The author does not encourage or support unauthorized access, exploitation, disruption, or attacks against systems, networks, applications, or data you do not own or have explicit permission to test. The developer is not responsible for misuse of this project or damage resulting from unauthorized deployment or use.

Use responsibly. Stay ethical. Test only with authorization.
Built for Cybersecurity Research & Education